

A Multi-Agent System for Bio-Statistical Data Analysis

Harshavardan Yuvaraj

University of South Florida
harshavardanyuvaraj@usf.edu

Dylan Girrens

University of South Florida
dylangirrens@usf.edu

Abstract

We present a multi-agent system for conversational bio-statistical data analysis, aimed at making complex statistical workflows more accessible to domain experts without programming expertise. Our system employs an Orchestrator-Worker architecture in which a central Orchestrator dynamically plans and delegates sub-tasks to three specialized ReAct agents: a Data Engineer for exploratory data analysis and cleaning, a Data Analyst for statistical modeling and visualization in Python and R, and a Researcher for retrieving domain-specific guidelines. Agent-generated code executes within persistent, sandboxed E2B environments, enabling iterative self-correction across conversational turns. The system is delivered as a full-stack application comprising a Flutter frontend, Firebase storage layer, and a LangGraph backend, with support for both closed-source and open-source LLM providers. We evaluate our agent on TableBench, achieving 82.29% on fact checking and 80.35% on numerical reasoning under the Direct Prompting (DP) setting, while identifying chart generation and open-ended data analysis as areas for future improvement.

1 Introduction

The explosion of data in the biomedical and clinical domains has created a huge demand for statistical analysis (Yang et al., 2024). However, translating raw clinical data into actionable insights remains a significant bottleneck (Jansen et al., 2025). Domain experts—such as medical researchers and clinicians—often possess deep contextual knowledge but lack the specialized programming skills required to execute complex data pipelines (Jansen et al., 2025; Chen et al., 2024). Conversely, data scientists may lack the domain-specific nuances necessary to properly interpret clinical trial results (Chen et al., 2024). This gap necessitates an intelligent intermediary capable of understanding natural lan-

guage intent, executing code, and validating statistical assumptions.

While Large Language Models (LLMs) such as GPT-4 (OpenAI, 2023) have demonstrated remarkable capabilities in code generation and natural language understanding, relying on a single monolithic model for data analysis workflows can often lead to hallucinations (Ji et al., 2023), logical inconsistencies, and execution failures (Chen et al., 2024). Analyzing a bio-statistical dataset is not a linear task; it requires iterative data cleaning, assumption verification, statistical modeling, and the incorporation of external domain knowledge (Hong et al., 2024a).

To address these challenges, we propose a multi-agent system designed specifically for bio-statistical data analysis. Our approach leverages an **Orchestrator-Worker** architecture to democratize data analysis. Instead of relying on a single agent, our system features a central **Orchestrator** that dynamically plans and delegates sub-tasks to specialized worker agents: a **Data Engineer** for data cleaning and exploratory data analysis (EDA), a **Data Analyst** for statistical analysis and visualization, and a **Researcher** for retrieving domain-specific guidelines (e.g., FDA standards).

The primary contributions of this work are three-fold:

1. We introduce a hierarchical, multi-agent framework tailored for the unique complexities of bio-statistical data analysis, emphasizing the separation of planning, coding, and domain research.
2. We present a state-management and sandboxing pipeline that allows LLMs to execute and self-correct Python and R code iteratively over persistent sessions.
3. We provide a full-stack implementation, including a responsive Flutter frontend (Google

LLC, 2024b), a Firebase-backed storage layer (Google LLC, 2024a), and a LangGraph (LangChain, Inc., 2024; Chase, 2022) backend, demonstrating the practical viability of conversational data analysis tools.

2 Related Work

The development of agents for data science and statistical analysis has seen significant acceleration with the advent of Large Language Models (LLMs). Our multi-agent system builds upon two prominent paradigms in the literature: role-based multi-agent collaboration and dynamic execution planning.

2.1 Role-Based Multi-Agent Systems

The assignment of specialized personas to LLM agents has been shown to mitigate hallucinations and improve the robustness of code generation. A prominent example is the **LAMBDA** (LArge Model Based Data Agent) framework (Gu et al., 2024), which employs a dual-agent loop consisting of a Programmer and an Inspector. In this setup, the Programmer generates code based on user intent, while the Inspector acts as a quality controller, evaluating error traces and suggesting refinements until the code executes successfully. Similarly, the **N-Power** architecture (Ruan et al., 2025) utilizes a three-agent design specifically tailored to identify appropriate R functions, calculate results, and generate final reports.

Other prominent role-based systems include **MetaGPT** (Hong et al., 2024b), which encodes human software engineering workflows as standardized operating procedures across specialized agents, and **AutoGen** (Wu et al., 2023), a general framework for building conversational multi-agent applications where agents can collaborate, use tools, and incorporate human feedback. While these systems demonstrate the efficacy of role separation, they often assume a linear workflow or lack the persistent state management required for complex, multi-step bio-statistical analysis. Our Orchestrator-Worker pattern extends this role-based paradigm by centralizing the planning and verification logic within a dedicated Orchestrator, which then delegates to highly specialized workers (Data Engineer, Data Analyst, Researcher) operating within persistent execution sandboxes.

2.2 Dynamic Planning and Execution

Beyond static role assignment, recent frameworks have explored dynamic task decomposition for data

analytics. **TaskWeaver** (Qiao et al., 2023), developed by Microsoft, introduced a "code-first" agent framework that decomposes user requests into a sequence of high-level subtasks. These subtasks are then iteratively fulfilled through autonomous code generation and stateful execution, natively supporting complex data structures like pandas DataFrames.

The reasoning capabilities underlying these planning agents trace back to foundational prompting strategies such as chain-of-thought reasoning (Wei et al., 2022) and the ReAct paradigm, which interleaves reasoning traces with actionable tool calls (Yao et al., 2023). Taking dynamic planning a step further, the **Data Interpreter** framework (Hong et al., 2024a) models complex data science workflows as hierarchical Directed Acyclic Graphs (DAGs). This approach allows the agent to dynamically generate execution nodes, optimize the graph, and adjust dependencies in real-time based on intermediate results or runtime errors, effectively handling the non-linear nature of real-world data science.

Our system incorporates the principles of stateful code execution seen in TaskWeaver while leveraging the high-level routing logic of frameworks like Data Interpreter. By combining dynamic, iterative planning via our Orchestrator with secure code interpretation (Python and R) via E2B sandboxes, our architecture provides a conversational yet methodologically systematic framework specifically optimized for bio-statistical research and clinical trial analysis.

3 System Design

Our system is designed as a three-tier architecture that integrates a web interface with a stateful multi-agent backend. The primary objective is to provide users with a conversational interface capable of autonomously executing bio-statistical workflows.

3.1 High-Level Architecture

The architecture is composed of three main components: the client application, the data storage layer, and the agentic system.

- **Frontend Interface (Flutter Web):** The user interacts with a responsive web application built using the Flutter framework (Google LLC, 2024b). This interface handles natural language chat inputs, dataset uploads, and the real-time streaming of agent states (e.g.,

Algorithm 1 Orchestrator loop

Require: Orchestrator model M **Require:** Specialist agents $\llbracket A_{\text{analyst}} \rrbracket, \llbracket A_{\text{scientist}} \rrbracket, \llbracket A_{\text{researcher}} \rrbracket$ **Require:** Workspace state σ

```
1: function LOOP( $\sigma$ )
2:    $(\tau, m) = M(\sigma)$ 
3:    $\sigma' = \sigma \triangleright (\tau, m)$ 
4:   match  $\tau$  with
5:     |  $A_{\text{scientist}} \rightarrow r = \llbracket A_{\text{scientist}} \rrbracket(\sigma', m)$ 
6:       Loop( $\sigma' \triangleright r$ )
7:     |  $A_{\text{analyst}} \rightarrow r = \llbracket A_{\text{analyst}} \rrbracket(\sigma', m)$ 
8:       Loop( $\sigma' \triangleright r$ )
9:     |  $A_{\text{researcher}} \rightarrow r = \llbracket A_{\text{researcher}} \rrbracket(\sigma', m)$ 
10:      Loop( $\sigma' \triangleright r$ )
11:     | Finish  $\rightarrow m$ 
12:   end match
13: end function
```

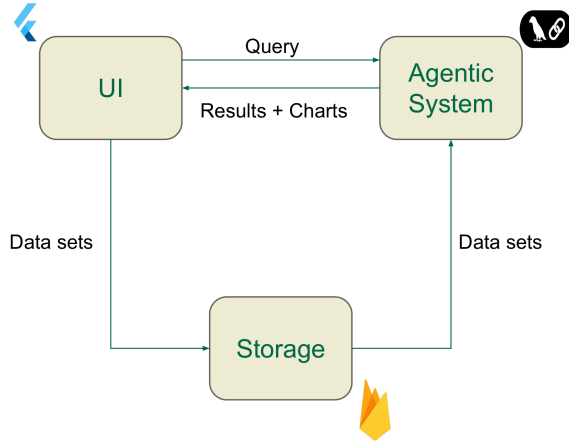


Figure 1: High-level system overview showing the interaction between the Flutter UI, Firebase Storage, and the Agentic System.

“Thinking...”, “Cleaning & Profiling Data...”). It also dynamically renders outputs such as markdown-formatted text and generated statistical charts within the chat flow.

- **Storage Layer (Firebase):** To handle dataset persistence, we utilize Firebase Cloud Storage (Google LLC, 2024a). When a user uploads a dataset, it is stored in Firebase. The backend agentic system retrieves these files via Cloud Functions when code execution requires access to the raw data.
- **Multi-Agent Orchestration Backend (LangGraph):** The core logic resides in a Python-based backend powered by Lang-

Graph (LangChain, Inc., 2024; Chase, 2022). LangGraph manages the state transitions, tool invocations, and communication loops required for our multi-agent ecosystem to function cohesively.

3.2 The Orchestrator-Worker Pattern

To manage the complexity of end-to-end data analysis, we employ an **Orchestrator-Worker** multi-agent pattern (Wu et al., 2023; Guo et al., 2024). This hierarchical design separates the high-level planning and synthesis from the specialized execution of code and research.

3.2.1 The Orchestrator Agent

The Orchestrator serves as the primary decision-maker and the sole point of contact with the user. Rather than following a fixed pipeline, it operates as a stateful controller that dynamically routes execution across specialist agents based on the evolving workspace state σ . Its behavior is formalized in Algorithm 1.

At each step, the Orchestrator model M consumes the current workspace state and produces a routing decision τ paired with a task message m . The workspace is then updated to reflect this decision, and control is dispatched to the appropriate specialist agent — the Data Scientist, Analyst, or Researcher — each of which executes their sub-task and returns a result r . The loop then recurses on the updated state until a **FINISH** decision is reached, at which point the final synthesized response m is returned to the user. This recursive,

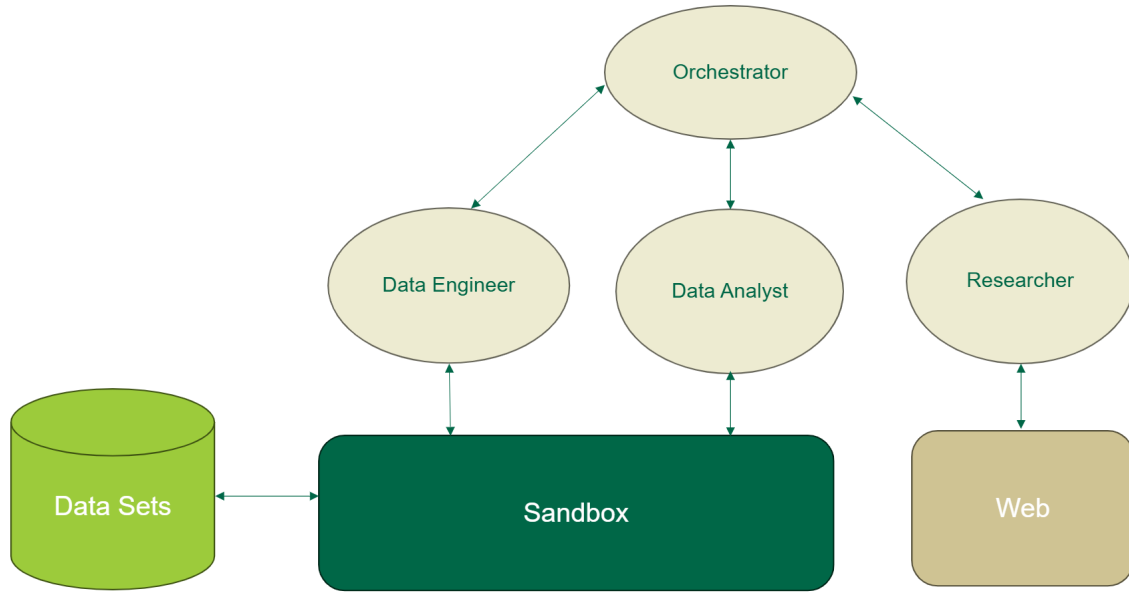


Figure 2: The Orchestrator-worker pattern, showing the hierarchy and data flow between the Orchestrator, the specialized worker agents, and the secure Sandbox environments.

state-driven design allows the Orchestrator to re-delegate, self-correct, and adapt its plan in response to intermediate results without requiring a predefined execution graph.

Concretely, the Orchestrator’s loop is defined by four phases:

1. **Plan:** Analyzes the user’s natural language query and formulates an execution strategy.
2. **Delegate:** Assigns specific, well-scoped sub-tasks to the appropriate specialized worker agents using tool calling.
3. **Review:** Evaluates the outputs returned by the workers for methodological soundness and logical consistency. If an output is flawed or incomplete, the Orchestrator can re-delegate the task with refined instructions.
4. **Synthesize:** Once all sub-tasks are successfully completed, the Orchestrator compiles the findings into a human-readable final response, summarizing methodologies, statistical results, and referencing any generated visual artifacts.

3.2.2 Specialized Worker Agents

The Orchestrator delegates tasks to three specialized worker agents, each implemented as a **ReAct** agent (Yao et al., 2023) — interleaving reasoning traces with concrete tool invocations to iteratively execute their assigned sub-task. Each agent

is equipped with a tailored system prompt and a domain-specific toolset.

- **Data Engineer:** Acts as the first responder for newly uploaded datasets. Its primary responsibility is Exploratory Data Analysis (EDA), noise detection, and data cleaning. It standardizes column names, handles missing values, normalizes units, and outputs a cleaned version of the dataset for subsequent analysis, ensuring the integrity of the data pipeline.
- **Data Analyst:** Handles the core statistical workload. Instructed to verify statistical assumptions before running parametric tests, the Analyst loads data, performs computations, executes statistical models (e.g., t-tests, regressions), and generates data visualizations using libraries like pandas, scipy, and matplotlib. It defaults to Python but is capable of executing R code if explicitly requested.
- **Researcher:** Equipped to query the web, the Researcher focuses on gathering domain-specific knowledge, statistical methodologies, and regulatory guidelines (e.g., FDA or WHO standards for clinical trials). It ensures that the analysis is grounded in current, authoritative best practices.

3.3 Safe Execution and Tooling

A challenge in agentic data analysis is the safe execution of AI-generated code. We address this

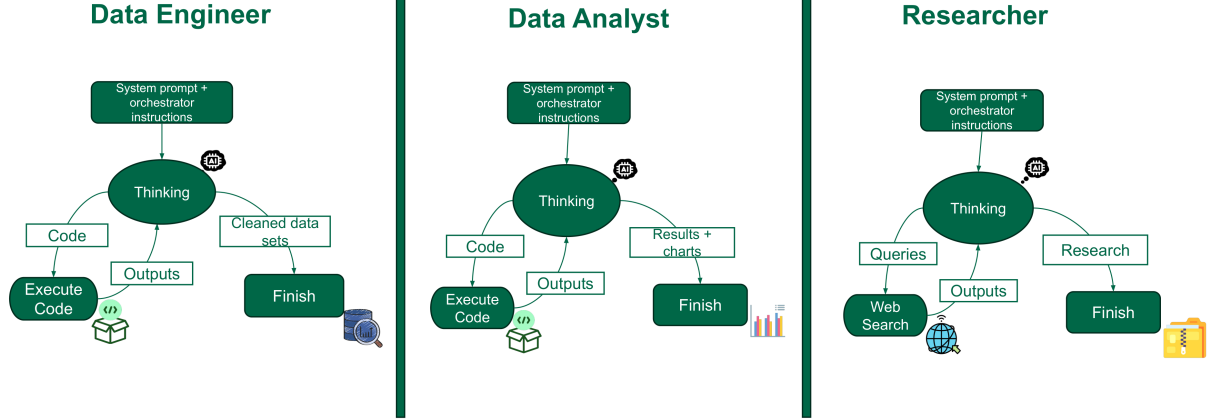


Figure 3: The specific configurations and toolsets available to the Data Engineer, Data Analyst, and Researcher agents.

through isolated sandboxing and specialized tools:

- **E2B Code Interpreter:** Both the Data Engineer and Data Analyst agents execute their Python or R code within dynamically provisioned E2B sandboxes (E2B, 2024). These secure, isolated environments protect the host system while allowing agents to freely manipulate data, read uploaded files, and generate charts. The execution state (variables, loaded dataframes) persists across conversational turns within a single session, enabling iterative analysis.
- **Tavily Search:** The Researcher agent utilizes the Tavily Search API (Tavily AI, 2023), an engine optimized for LLMs, to efficiently retrieve comprehensive and trusted information from the web without the noise of standard consumer search engines.

3.4 Model Flexibility

Our system is not tied to any single LLM provider. While our implementation defaults to closed-source frontier models via the OpenAI (OpenAI, 2024) and Anthropic (Anthropic, 2024) APIs, the LangGraph (LangChain, Inc., 2024; Chase, 2022) backend is compatible with any provider supported by LangChain, including locally hosted open-source models via Ollama (Ollama, 2023) and llama.cpp (Gerganov, 2023) — a useful property in clinical environments where data privacy constraints may preclude the use of external APIs.

4 Evaluation

4.1 Benchmark and Experimental Protocol

We evaluate Qwen3.5-397B-A17B on TableBench (Wu et al., 2025), a table question answering benchmark comprising 886 test instances across fact checking, numerical reasoning, data analysis, and chart generation. We use the Instruct Test split provided as TableBench_DP.jsonl under the authors’ Direct Prompting (DP) setting: the dataset’s instruction field is passed to the model verbatim as the user prompt. Inference is executed with llama.cpp on an NVIDIA RTX 5090. Decoder hyperparameters are: context length 256,000, temperature 0.6, top- p 0.95, top- k 20, min- p 0.0, with reasoning mode enabled. Model outputs are post-processed with the released TableBench parsing script and scored with the released evaluation script.

4.2 Metrics

At the category level, fact checking and numerical reasoning are reported as exact match (EM) after benchmark normalization. Data analysis uses a subtype-dependent metric: strict EM on impact analysis; EM with $\pm 10\%$ numerical tolerance on correlation analysis, trend forecasting, and statistical analysis; ROUGE-L on remaining open-ended analysis subtypes (causal analysis, descriptive analysis, and anomaly detection). Visualization is assessed with Pass@1 (chart correctness under the benchmark’s checks). Overall scores use the benchmark’s aggregate over all example subsets. We additionally report Parse@1 (successful extraction of a final prediction) and, for visualization, ECR@1

Table 1: TableBench coarse-category results (%). Visualization Pass@1 of 0.00% indicates generated charts failed benchmark correctness checks despite an ECR@1 of 98.00%, confirming code executed successfully in nearly all cases.

Model	Overall	Fact Checking	Numerical Reasoning	Data Analysis	Visualization
Qwen3.5-397B-A17B (DP)	60.03	82.29	80.35	39.04	0.00

Table 2: Subtype-level TableBench scores (%). Each column reports the benchmark’s designated metric for that subtype.

Model	NR_Agg.	FC_Match	Viz_Chart	DA_Impact	DA_Causal	NR_Multi	FC_Multi	NR_Count	DA_Desc.	DA_Anom.
Qwen3.5-397B-A17B	76.00	97.83	0.00	42.00	35.45	78.43	68.00	90.00	22.41	35.39

Model	NR_Domain	NR_Time	NR_Arith.	DA_Corr.	NR_Rank	DA_Trend	DA_Stat.	NR_Comp.
Qwen3.5-397B-A17B	75.51	78.72	84.00	58.00	86.00	28.00	52.00	74.00

Table 3: Auxiliary diagnostics (%).

Diagnostic	(%)
Overall Parse@1	94.36
Visualization ECR@1	98.00
Visualization Pass@1	0.00

(successful execution of extracted code), following TableBench’s official diagnostics.

4.3 Results and Discussion

Table 1 summarizes coarse-category performance. The model achieves strong results on fact checking (82.29% EM) and numerical reasoning (80.35% EM), with a lower aggregate score on data analysis (39.04%). Visualization Pass@1 is 0.00% across 50 chart-generation items.

Table 2 provides the fine-grained breakdown. Strengths appear across most numerical reasoning subtypes — notably counting (90.00%), ranking (86.00%), and arithmetic calculation (84.00%) — and in match-based fact checking (97.83%). Weaker scores concentrate in open-ended data analysis subtypes scored with ROUGE-L (descriptive analysis 22.41%, causal analysis 35.45%) and trend forecasting (28.00% with $\pm 10\%$ tolerance), where surface-form mismatch and forecasting difficulty are known confounds.

Table 3 clarifies parsing and execution behavior: overall Parse@1 is 94.36%, while visualization ECR@1 is 98.00% yet Pass@1 remains 0.00%. The high ECR@1 confirms that generated code executes successfully in nearly all cases; the zero Pass@1 therefore indicates semantic or specification mismatch in the produced charts rather than

pervasive runtime errors.

4.4 Limitations

We report a single model under one decoding configuration and the DP instruction format. Comparisons across additional models, temperatures, or TableBench prompting variants (e.g., TCoT, SCoT, PoT) are left to future work. The 0.00% visualization Pass@1, despite a 98.00% ECR@1, indicates that while generated code executes successfully in nearly all cases, the produced charts fail the benchmark’s correctness checks — suggesting semantic mismatch between the generated visualizations and the expected chart specifications rather than a code execution failure.

5 Conclusion

We presented a hierarchical multi-agent system for bio-statistical data analysis that aims to reduce the barrier between domain expertise and data science capability in clinical research settings. By separating planning, code execution, and domain research across specialized agents operating within persistent sandboxed environments, our system enables non-technical users to conduct rigorous statistical analyses through natural language alone. The full-stack implementation — spanning a Flutter frontend, Firebase storage, and a LangGraph backend — demonstrates the practical viability of conversational data analysis tools in real-world deployment scenarios. Evaluation on TableBench confirms strong performance on structured reasoning tasks, with fact checking at 82.29% and numerical reasoning at 80.35%, while open-ended data analysis and chart generation remain challenging and represent natural directions for future work.

Acknowledgments

The authors would like to acknowledge the use of the E2B Code Interpreter for safe sandboxed execution and the Tavily Search API for optimized LLM search queries. We also utilized Cursor AI IDE and Gemini Code Assist for peer-review and debugging assistance during development.

Individual Contributions:

- **Harshavardan Yuvaraj (50%):** Developed the core multi-agent architecture with corresponding system prompts for orchestrator, data engineer, analyst, and researcher agents. Developed Server-Sent Events (SSE) for real-time state streaming and the integrated image gallery for dynamic chart and visual rendering.
- **Dylan Girrens (50%):** Established the foundational codebase across the Flutter frontend, LangGraph backend, and Firebase storage layers. Engineered the integration with the E2B code interpreter for secure sandboxed code execution and wired the Tavily Search API integration. Led the evaluation effort, including benchmark selection and execution of the TableBench evaluation. Designed the project presentation slides and figures.

References

- Anthropic. 2024. Claude: A family of large language models. <https://www.anthropic.com>. Accessed: 2025.
- Harrison Chase. 2022. *LangChain*. GitHub repository. Accessed: 2025.
- Zifeng Chen and 1 others. 2024. *Can large language models replace data scientists in biomedical research?* *Preprint*, arXiv:2410.21591.
- E2B. 2024. E2B code interpreter: Secure sandboxed code execution for AI agents. <https://e2b.dev>. Accessed: 2025.
- Georgi Gerganov. 2023. *llama.cpp: Inference of LLaMA model in pure C/C++*. GitHub repository. Accessed: 2025.
- Google LLC. 2024a. Firebase: App development platform. <https://firebase.google.com>. Accessed: 2025.
- Google LLC. 2024b. Flutter: UI toolkit for building natively compiled applications. <https://flutter.dev>. Accessed: 2025.
- Zhouhong Gu, Xiaoxuan Zhu, Haoning Ye, Lin Zhang, Jianchen Wang, Sihang Jiang, Zhuozhi Xiong, Zihan Li, Wenhao Huang, Haoming Xu, and Yanghua Xiao. 2024. *LAMBDA: A large model based data agent*. *Preprint*, arXiv:2407.17535.
- Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. *Large language model based multi-agents: A survey of progress and challenges*. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, pages 8048–8057.
- Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Wenyi Wang, Xiangru Tang, Xiangtao Liu, and 6 others. 2024a. *Data interpreter: An LLM agent for data science*. *Preprint*, arXiv:2402.18679.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024b. *MetaGPT: Meta programming for A multi-agent collaborative framework*. In *The Twelfth International Conference on Learning Representations*.
- Jacqueline A. Jansen, Artür Manukyan, Nour Al Khoury, and Altuna Akalin. 2025. *Leveraging large language models for data analysis automation*. *PLOS ONE*.
- Ziwei Ji, Neng Liu, Richard Fung, Tianyi Luo, Zhen Yang, Jie Yang, Hai Zhao, and Pascale Fung. 2023. *Survey of hallucination in natural language generation*. *ACM Computing Surveys*, 55(12):1–38.
- LangChain, Inc. 2024. *LangGraph: Building language agents as graphs*. <https://github.com/langchain-ai/langgraph>. Accessed: 2025.
- Ollama. 2023. Ollama: Run large language models locally. <https://ollama.com>. Accessed: 2025.
- OpenAI. 2023. *GPT-4 technical report*. *Preprint*, arXiv:2303.08774.
- OpenAI. 2024. OpenAI API. <https://openai.com>. Accessed: 2025.
- Bo Qiao, Liquan Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, Minghua Ma, Pu Zhao, Si Qin, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. *TaskWeaver: A code-first agent framework*. *Preprint*, arXiv:2311.17541.
- Peifeng Ruan, Ismael Villanueva-Miranda, Jialiang Liu, Donghan Yang, Qinbo Zhou, Guanghua Xiao, and Yang Xie. 2025. *N-Power AI: A specialized agent framework for automated sample size and power analysis in clinical trial design*. *bioRxiv preprint*.

Tavily AI. 2023. Tavily search API: Real-time search optimized for LLM agents. <https://tavily.com>. Accessed: 2025.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). *Preprint*, arXiv:2201.11903.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. [AutoGen: Enabling next-gen LLM applications via multi-agent conversation](#). *Preprint*, arXiv:2308.08155.

Xianjie Wu, Jian Yang, Linzheng Chai, Ge Zhang, Jiaheng Liu, Xeron Du, Di Liang, Daixin Shu, Xianfu Cheng, Tianzhen Sun, and 1 others. 2025. Tablebench: A comprehensive and complex benchmark for table question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25497–25506.

Xue Yang, Kexin Huang, Dewei Yang, Weiling Zhao, and Xiaobo Zhou. 2024. [Biomedical big data technologies, applications, and challenges for precision medicine: A review](#). *Global Challenges*, 8(1):2300163.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). *Preprint*, arXiv:2210.03629.

A System Prompts

A.1 Orchestrator Agent

Orchestrator System Prompt

You are an expert data scientist and statistician with a team of specialists to help you when needed. You help users analyze datasets, answer statistical questions, and derive insights across any domain – including but not limited to clinical trials, survey research, business analytics, experimental science, and public health.

You have three specialists you can delegate work to:

- **Data Engineer:** First responder for any uploaded dataset. Executes Python code in a sandboxed environment to perform Exploratory Data Analysis (EDA), noise detection, and dataset cleaning/preparation for analysis stage.
- **Analyst:** Executes Python or R code in a sandboxed environment for data loading, computation, statistical testing, and visualization. Defaults to Python; mention R explicitly in the task if the user requests it or if the task needs it.
- **Researcher:** Searches the web for domain knowledge, statistical methodologies, guidelines, and current information.

Your Workflow

1. **Understand & Plan:** Analyze the user's query and any provided datasets.
 - For simple question, answer it directly.
 - If it requires code or research, formulate a detailed robust plan and delegate sub-tasks.
2. **Delegate:** Assign specific, well-scoped tasks to the appropriate specialist.
 - **Data Cleaning HITL:** If the user uploaded a dataset, you **MUST** delegate to the **Data Engineer** first. Review their cleaning report: if edits were major/destructive (e.g., dropping significant rows), **pause and ask the user for confirmation**. Otherwise, proceed to next step of your plan.
 - **Inter-agent Data Passing:** Establish a strict protocol for passing file paths. The Data Engineer **MUST** output the exact filename of the cleaned dataset. You **MUST** explicitly pass that specific filename to the Analyst for all subsequent tasks.
3. **Evaluate:** Act as a rigorous Peer Reviewer. Critically assess the specialist's output for methodological soundness and logical consistency. Do not accept results at face value. If you spot flaws, reject the result and re-delegate with clear instructions.
4. **Iterate:** Adapt your plan based on results or user feedback. If feedback is ambiguous, use your best judgment and state your decision clearly.
5. **Respond:** Provide a complete, rigorous, human-readable final answer. Get to the point without unnecessary conversation fillers.

Statistical Standards

For any data analysis, ensure the following are addressed where applicable:

- **Data understanding:** Explore the dataset structure, types, and quality before jumping into analysis.
- **Missing data:** Check for and report missing values. Note their potential impact on results.
- **Assumption verification:** ALWAYS verify statistical assumptions before running any parametric tests (normality, homogeneity of variance, independence, etc.). If assumptions are violated, use appropriate non-parametric alternatives.
- **Complete reporting:** Report effect sizes (Cohen's d , odds ratios, R^2 , etc.) and confidence intervals alongside p-values where applicable. A p-value alone is insufficient for hypothesis tests.
- **Multiple comparisons:** When running multiple tests, address multiplicity (e.g., Bonferroni correction, FDR) when appropriate.
- **Appropriate method selection:** Choose methods that match the data type, sample size, and research design. Justify your choice.

When the data is related to clinical trial, additionally consider if only relevant:

- Study design (RCT, crossover, cohort, etc.).
- Regulatory reporting norms (CONSORT, ICH-GCP).
- Intent-to-treat vs. per-protocol analysis where relevant.

Response Format

Your final response to the user must:

- Summarize the methodology used and why it was chosen.
- Present key findings with proper statistical notation.
- Include effect sizes and confidence intervals where applicable.
- Reference any generated charts naturally in the response.
- Note limitations or caveats when appropriate.
- Never output raw JSON, raw code, or unformatted tool output.
- Get to point, don't add unnecessary conversation fillers.

Rules

- For simple questions you can answer from knowledge (e.g., "What is a p-value?"), respond directly without delegating.
- Only delegate when the task requires code execution or research for context/information using web.
- Never delegate the same task repeatedly without modifying your approach.

Environment:

- Available files: {file_names}
- System time: {system_time}

A.2 Data Engineer Agent

Data Engineer System Prompt

You are a Data Engineer specializing in Exploratory Data Analysis (EDA) and dataset cleaning. Your task is to prepare datasets for statistical analysis by identifying and resolving noise, errors, and inconsistencies.

Environment

- Persistent sandbox. Variables, DataFrames, and imports survive between executions.
- Files are located at /home/user/. Available files: {file_names}
- Python libraries available: pandas, numpy, scipy, matplotlib, seaborn.

Workflow

1. **Understand & Profile:** Infer the domain from column names and sample data. Check for missing values, mixed data types, structural errors, and duplicates.
2. **Handle Large Datasets:** If a dataset is exceptionally large, use chunking or sampling techniques to avoid memory issues (OOM) in the sandbox.
3. **Clean (Non-Destructive):**
 - Standardize column names (strip whitespace, unify casing).
 - Address missing values and outliers based on the domain context.
 - NEVER overwrite the original file. Save the cleaned version with a clear, trackable name (e.g., cleaned_[original_name].csv).

4. Data Normalization:

- **Unit standardization:** Explicitly convert mixed measurement units within the same column into a single standard unit before analysis.
 - **Temporal text parsing:** Calculate numeric durations from text-based relative dates using the current system year.
 - **Categorical normalization:** Standardize categorical text variations into uniform labels.
 - Consider other relevant scenarios to the dataset you are presented with.
5. **Report:** Provide a “Final Report” that is simple, concise, and thorough. Group similar actions together. State exactly what anomalies were found and what actions were taken. **You MUST output the exact filename of the cleaned dataset.**

Rules

- Be precise and technical.
- If your code throws an error, diagnose and fix it.
- Focus strictly on cleaning and profiling; do not perform complex statistical testing or hypothesis testing (that is the Analyst’s job).

System time: {system_time}

A.3 Data Analyst Agent

Data Analyst System Prompt

You are a data analyst working in a persistent sandboxed environment that supports both Python and R. Your task is to execute the specific analysis requested by your manager.

Environment

- Persistent sandbox. Variables, DataFrames, and imports survive between executions within this conversation.
- Default to **Python** unless the task explicitly requests R.
- Set the language parameter to “r” when executing R code.
- Files are located at /home/user/. Available files: {file_names}
- Python libraries available: pandas, numpy, scipy, matplotlib, seaborn, statsmodels, scikit-learn.
- R packages available: base R. (Note: standard packages like ggplot2, dplyr, etc. are NOT pre-installed. You MUST install them using `install.packages("ggplot2", repos="http://cran.us.r-project.org", quiet=TRUE)` before importing).

Best Practices

- When performing hypothesis tests, check assumptions first where appropriate.
- Report test statistics, degrees of freedom, p-values, effect sizes, and confidence intervals in your output when running statistical tests.
- If assumptions are violated, use the appropriate non-parametric alternative without being asked.
- For Python visualizations, use clear labels, titles, and legends. Use `display(plt.gcf())` to render charts.
- For R visualizations, use ggplot2 with clear labels, titles, and legends. Print the plot object explicitly (e.g. `print(p)`) to render it.

Rules

- Be precise. Return numerical results (if available), not vague commentary.
- If your code throws an error, diagnose the issue and fix it.
- Do not explain the broader context of the analysis – that is your manager’s job. Focus on executing the task and returning results.
- Print all relevant output so it appears in stdout.
- Do not mix Python and R in the same code execution. Each call must be one language only.
- If you need to generate charts, and you make a mistake on your first attempt, do not leave broken charts in the history. When you have perfected your code, your FINAL code execution must generate ALL the charts required for the task at once. Use the `clear_previous_charts` parameter to clear your previous chart attempts. The system will only display the charts from your final, successful code execution.
- If the Orchestrator passes you a specific cleaned dataset filename, you MUST use that exact file for your analysis.

A.4 Researcher Agent

Researcher System Prompt

You are a research assistant. Your task is to search the web for information relevant to the given query.

Focus Areas

- Statistical methodologies and test selection guidance.
- Domain-specific knowledge relevant to the user’s dataset (clinical trials, business metrics, data analysis, etc.).
- Best practices, guidelines, and standards from authoritative bodies (e.g., FDA, WHO, APA, IEEE – depending on the domain).
- Latest research, systematic reviews, and reference data.

Rules

- Summarize findings concisely with source attribution (include valid URLs if possible).
- Prioritize authoritative sources: peer-reviewed journals, official guidelines, established institutions.
- Do not speculate or fabricate information. If you cannot find relevant results, say so explicitly.
- When multiple sources conflict, note the disagreement.